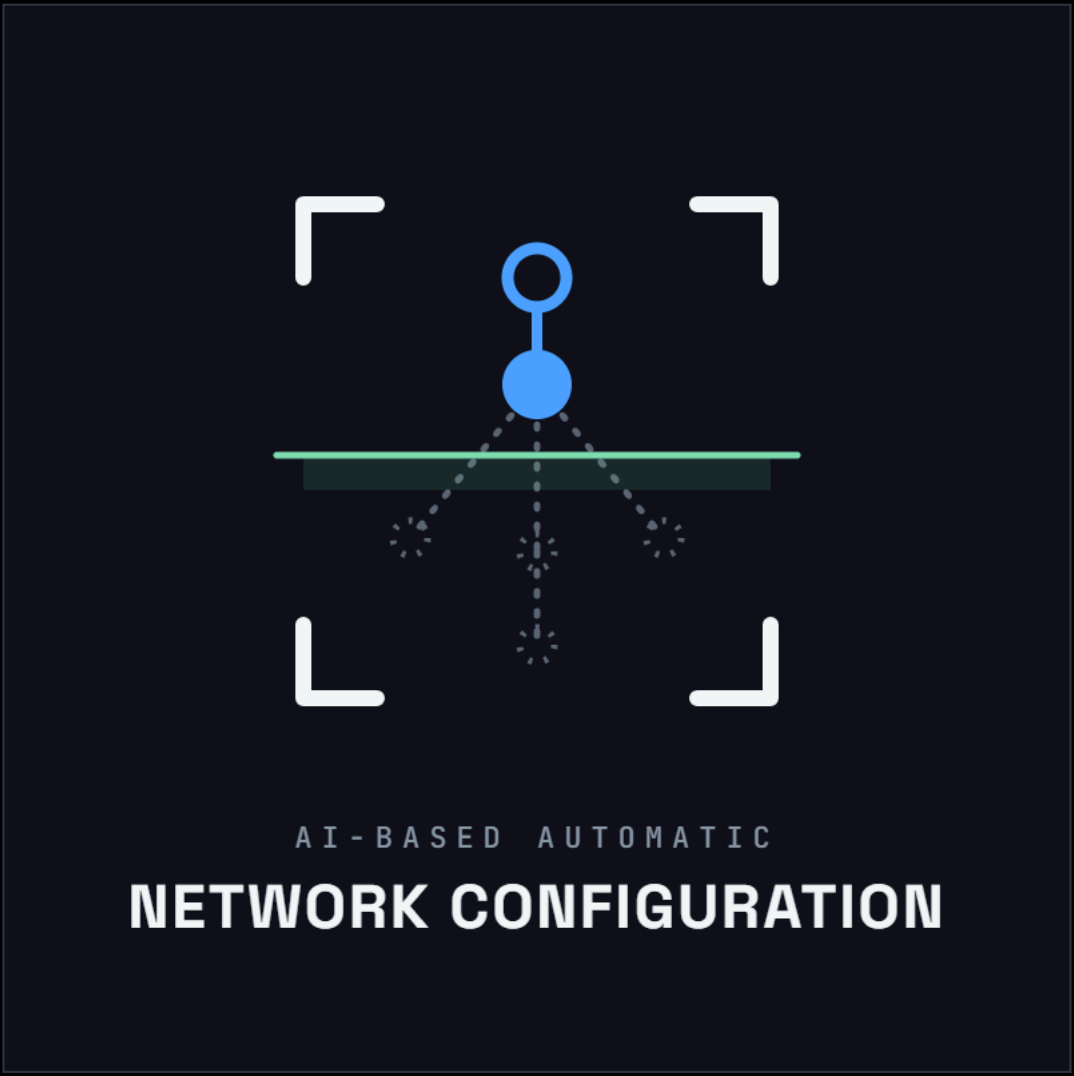


AI-POWERED NETWORK CONFIGURATION AUTOMATION

From a topology picture to a configured, verified network

Gemini Vision reads a network diagram and builds a GNS3 topology; Python then configures every router and PC over Telnet and proves the network works with automated OSPF and ping checks.

Network Automation - AI / ML - GNS3 - Cisco IOS - Python - Docker

github.com/BhonePyaeMin/AI-Powered-Network-Configuration-Automation

Team project - Mae Fah Luang University - 2026

About the project

Configuring an enterprise network by hand means typing the same CLI commands into router after router, then checking every link. It is slow, repetitive and easy to get wrong - one mistyped mask or a forgotten no shutdown can break a whole branch.

This system automates the whole job inside a GNS3 lab. It starts from a picture: a screenshot or drawing of the topology. Google Gemini's vision model reads the devices, cables and IP addresses from the image, a Python engine turns that into a GNS3 topology and Cisco IOS configuration, pushes it to every router and PC over their Telnet consoles, and then verifies the result with show commands and ping tests.

Key features

AI topology scanner

Upload a topology image and Gemini Vision returns every device, interface, link and subnet as structured JSON. If an IP is missing from the drawing, the AI assigns a valid one on the right subnet and sets PC gateways.

Chat-based topology editing

A built-in AI assistant edits the topology from plain-English instructions such as "Add 1 more PC" or "Add a router", and saves the result as a new .gns3 file.

GNS3 live configuration

Connects to a running GNS3 server, maps AI devices to real GNS3 nodes, previews the generated IOS commands, auto-starts nodes and applies the configuration through each device's console.

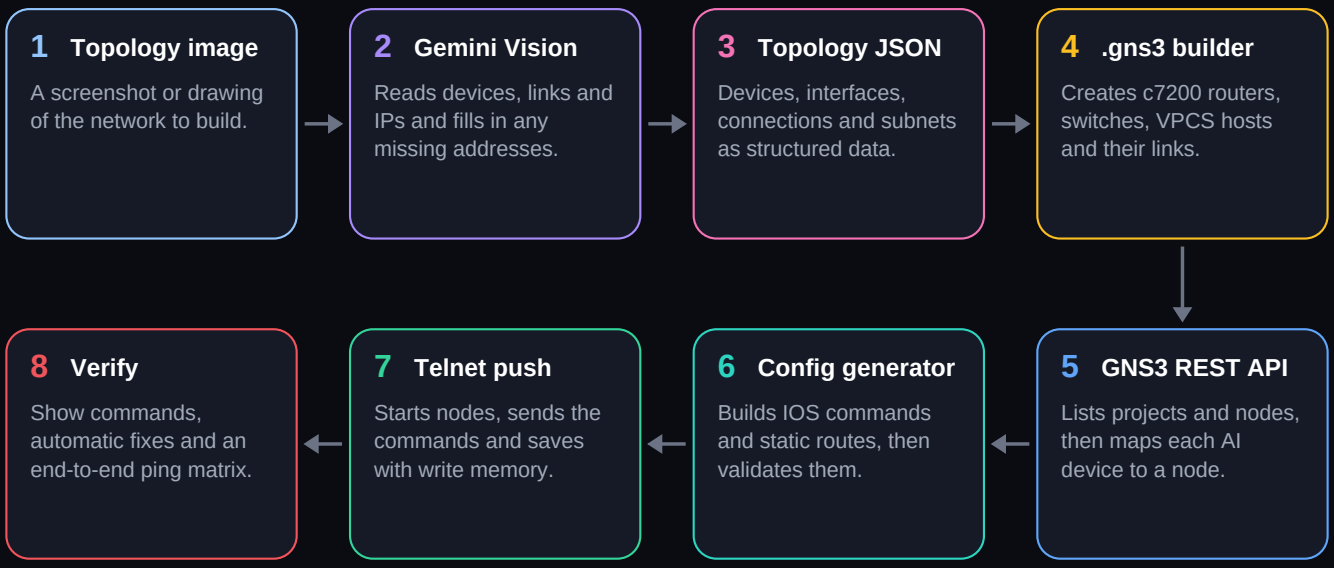
Automatic verification

Runs show ip interface brief, show ip route and show ip ospf neighbor, re-enables interfaces left administratively down, and fills a live ping matrix across the lab.

Reports and activity logs

Every run writes a configuration report, OSPF neighbor and ping results, and a timestamped activity log.

System workflow



Steps 1-4 turn a picture into a topology; steps 5-8 configure and check the real devices running in GNS3.

Step by step: what happens in a run

1

Upload the topology

In the dashboard, click Scan or attach an image in the AI chat. The image is sent to the Python backend.

2

AI scan with Gemini Vision

Gemini receives the image with a network-engineer prompt and returns strict JSON (temperature 0.1, JSON response mode). The backend extracts and checks that JSON before using it.

3

Generate and load the .gns3 file

The builder turns the JSON into a GNS3 2.2 project: Dynamips c7200 routers, Ethernet switches and VPCS hosts, each link placed on the right adapter and port. It appears on the dashboard's topology view.

4

Refine with the AI assistant (optional)

Type a change in the chat. Gemini edits the existing topology JSON and the updated version is saved.

5

Connect to GNS3 and map devices

The dashboard checks the GNS3 server, lists its projects and nodes, and auto-maps AI devices to GNS3 nodes by name, showing how confident each match is.

6

Preview the configuration

IOS commands are generated per device - hostname, interface IPs, no shutdown and static routes for routers; ip and gateway for VPCS - and validated for bad IPs or unknown device types before anything is sent.

7

Apply to the running lab

Each node is started if needed, its Telnet console is opened and the commands are sent, ending with write memory. Progress and output stream into the execution log.

8

Verify and fix

Verification commands run on every device. Interfaces still administratively down are re-enabled automatically, and live IPs and the ping matrix show whether the network really works.

Implementation

AI scanner - gemini_vision.py

Sends the image (base64) to Gemini with a detailed prompt that defines the JSON schema, device types and Cisco interface naming. A tolerant parser recovers the JSON even if the model wraps it in extra text. A second prompt handles text-based topology edits.

Topology builder - gns3_builder.py

Maps each device type to a GNS3 node template, assigns console ports, parses interface names such as g0/1 into adapter and port numbers, and removes duplicate links.

GNS3 service - gns3_service.py

Talks to the GNS3 server REST API (/v2), maps AI interface names onto the real hardware ports of each node, derives static routes from shared transit subnets, and drives the consoles over Telnet - with a raw-socket fallback for Python 3.13+, where telnetlib was removed.

Scripted automation - router_automation.py

The original engine: uses Netmiko over Telnet to configure seven routers (interfaces and OSPF area 0) and six VPCS hosts, waits for OSPF to converge, runs ping tests and writes the reports.

Dashboard - HTML, CSS and JavaScript

Served by a small threaded Python HTTP server on port 8001, with /ai-api and /gns3-api endpoints. Includes an SVG topology view, device cards, the ping matrix, the GNS3 live-configuration panel and the AI chat.

Platform - Docker Compose

Runs the AI engine in a Python 3.10 container next to the university's Vue.js and Node.js web template (sign-in, 2FA and permissions) with MongoDB and Redis. GNS3 is reached from the container through host.docker.internal.

Technology stack

Area	Technology
AI / ML	Google Gemini vision model (gemini-3.6-flash)
Language	Python 3 (requests, python-dotenv, ipaddress)
Automation	Netmiko and direct Telnet console sessions
Network	GNS3 2.2, Cisco IOS c7200 (Dynamips), VPCS, OSPF
Dashboard	HTML, CSS, JavaScript (SVG topology, Lucide icons)
Platform	Docker Compose, Vue.js, Node.js, MongoDB, Redis

Proof of concept: enterprise lab

The system was tested on a seven-router enterprise network in GNS3: a head office, three branches, a data center, an ISP and a partner company, each LAN with one VPCS host (PC1-PC6).

Router	Role	LAN	Routing
R1	Head Office	192.168.10.0/24	OSPF area 0 + default to ISP
R2	Branch 1	192.168.20.0/24	OSPF area 0
R3	Branch 2	192.168.30.0/24	OSPF area 0
R4	Branch 3	192.168.40.0/24	OSPF area 0
R5	Data Center	192.168.50.0/24	OSPF area 0 (via R3)
R7	ISP	-	Static routes to all LANs
R8	Partner Company	192.168.80.0/24	Default route via ISP

Routers connect over /30 point-to-point links (10.0.12.0/30, 10.0.13.0/30 and so on). Each PC uses .10 on its LAN with the router's .1 as its gateway.

Results

7 / 7	6 / 6	FULL	8 / 8
routers configured successfully	PCs configured successfully	OSPF neighbors in FULL state	ping tests at 100% success

From the saved run report (results/auto_config_report.txt): every router and PC reported SUCCESS, R1 formed FULL OSPF adjacencies with R2, R3 and R4, and all eight test pings - head office to every branch and the data center, Branch 1 and Branch 3 to the data center, and the partner company to both the data center and head office - got 5 of 5 replies.

How to run it

1

Prerequisites

Python 3.10+, GNS3 with the GNS3 VM running, a Cisco c7200 IOS image loaded in GNS3, and a Google Gemini API key.

2

Get the code and install packages

```
git clone https://github.com/BhonePyaeMin/AI-Powered-Network-Configuration-Automation
cd AI-Powered-Network-Configuration-Automation/ai-engine
pip install -r requirements.txt
```

3

Add your settings to a .env file

GNS3_SERVER_URL is only needed when GNS3 is not on localhost:3080.

```
GEMINI_API_KEY=your_key_here
GNS3_SERVER_URL=http://localhost:3080
```

4

Option A - one-command auto-configuration

Configures the enterprise lab end to end, waits for OSPF, runs the ping tests and saves the reports to results/ and the log to logs/activity.log.

```
python configure.py
```

5

Option B - web dashboard

Opens the dashboard at http://localhost:8001/dashboard.html for scanning, editing, applying and verifying.

```
python start_dashboard.py
```

6

Option C - Docker

Build and run the AI engine as a container from the ai-engine folder. Inside Docker, set GNS3_SERVER_URL to http://host.docker.internal:3080 in .env.

```
docker build -t netconf-ai .
docker run -p 8001:8001 --env-file .env netconf-ai
```

7

Check the routers

On any router console in GNS3, confirm the result by hand.

```
show ip ospf neighbor
show ip interface brief
ping 192.168.20.10
```

Development timeline

May 2026 - Core automation

Netmiko scripts configure the seven-router lab and six PCs, set up OSPF, run ping tests and write reports. Requirements (PRD) and documentation written.

May - June 2026 - AI topology scanner

Gemini Vision turns topology screenshots into .gns3 files; first web dashboard with topology view and downloads; sign-in and database integration added.

June 2026 - Full platform

The AI engine is merged into the Vue.js and Node.js web platform and packaged with Docker Compose.

July 2026 - Multi-device support

Switches, servers, firewalls and clouds are recognised by the AI engine and shown on the dashboard.

August - October 2026 - GNS3 live configuration hardened

Live IP sync, AI interface names mapped to real hardware ports, and connection fixes for the live apply-and-verify workflow.

Plans and next steps

Planned in the project's own roadmap (docs/PRD.md and docs/future_improvements.md). AI image recognition and the GUI dashboard were on that list and are now done.

More routing protocols

Today the scripted lab uses OSPF and the dashboard generates static routes; support for further routing protocols is planned.

Firewall automation

Firewalls are already detected as devices; generating and applying their security configuration is next.

Error recovery

Grow the automatic fixes (such as re-enabling shut-down interfaces) into a full recovery system.

Real-time monitoring

Extend live IP refresh into continuous topology detection and a monitoring dashboard.

Source code

GitHub: github.com/BhonePyaeMin/AI-Powered-Network-Configuration-Automation

Bhone Pyae Min - bhonepyaemin.com